# LESSON 30

# 打包 拆开




## 元组：把几个值打成一包

学完你能让函数一次交回两个结果，把名字和分数、横坐标和纵坐标成对地放在一起。

# 有些东西总是成对出现

地图上的一个点

**横 3，竖 5**

想让函数交回

**最低分和最高分**

一个点要两个数才说得清，一个学生有名字也有分数。第 15 课的 return 只交回一个值，可有时想一次交回两个。

元组能把几个值打成一包，当成一个东西传来传去，用的时候再拆开。

# 圆括号打包，逗号隔开

point.py

```
1 p = (3, 5)
2 print(p)
3 print(p[0], p[1])
4 x, y = p
5 print(x + y)
```

x, y = p 叫拆包：左边几个名字，就按顺序拆成几个。第 24 课的 a, b = b, a 用的也是它。

```
(3, 5)
3 5
8
```

# 猜一猜：元组里的数能改吗？

change.py

```
1 p = (3, 5)
2 p[0] = 9
3 print(p)
```

屏幕上

**TypeError**

报 TypeError：元组不支持给某一格重新赋值。元组打好包，就不能给哪一格重新赋值，比列表“牢”。

要换一个点，就整个重新写：p = (9, 5)。这样也不容易被别的代码不小心改掉。

# 小测：运行这两行，输出什么？

```
a, b = (4, 7)
print(b, a)
```

1 4 7

2 7 4

3 (7, 4)

4 报错

答对了！拆包后 a 是 4、b 是 7，先输出 b。

# 函数一次交回两个

low_high.py

```python
def low_high(nums):
    return min(nums), max(nums)
low, high = low_high([7, 2, 9, 4])
print("最低", low)
print("最高", high)
```

| 交回 | (2, 9) |
| --- | --- |
| low | 2 |
| high | 9 |

调用的地方左边写两个名字，就把这一包拆开：low 拿到 2，high 拿到 9。

# 列表里装着一对一对

pairs.py

```
students = [("小明", 90), ("小红", 85)]
for name, score in students:
    print(f"{name}：{score} 分")
```

for 后面写两个名字，每次拿出一对，顺手就拆开了：第一次 name 是小明，score 是 90。

```
小明：90 分
小红：85 分
```

# 小测：这段程序输出什么？

quiz.py

```
1 pairs = [("A", 1), ("B", 2)]
2 for name, score in pairs:
3     print(score)
```

1 A 和 B

2 1 和 2

3 ("A", 1) 和 ("B", 2)

答对了！每次拿出一对，score 是第二个：1、2。

# 例题：最低、最高和相差

stats.py

```python
def low_high(nums):
    return min(nums), max(nums)
n = int(input())
scores = []
for i in range(n):
    scores.append(int(input()))
low, high = low_high(scores)
print("最低", low, "最高", high)
print("相差", high - low)
```

两个值都拿到了，还能接着算：96 - 77 是 19。

```
最低 77 最高 96
相差 19
```

# 动手改一改

让函数多交回一个。

## 再交回平均分

在例题上改：函数改成一次交回三个值：最低、最高、平均分，平均分用 round 保留 1 位小数。最后多输出一行“平均 多少”。输入和例题一样，n 是 1 到 50 的整数。

输入 4、90、85、77、96，第三行

```
平均 87.0
```

输入 1、60，第三行

```
平均 60.0
```

交回几个，拆的时候左边就写几个名字。教室电脑：打开 `starter/05_stats_avg.py`

# 这节课常见的三种错误

划线的写法只阅读，不要照抄

| | | |
|---|---|---|
| ~~`p[0] = 9`~~ | **TypeError** | 不能给某一格重新赋值，整个重写 |
| ~~`x, y = (1, 2, 3)`~~ | **ValueError** | 左右个数要一样 |
| ~~`t = (5)`~~ | **不报错** | 这只是数 `5`；要写 `(5,)` |

元组靠逗号打包，不靠括号。只有一个值时，后面也要跟一个逗号。

# 找错：第 3 行报 ValueError

bug.py（只阅读）

```
1 def low_high_total(nums):
2     return min(nums), max(nums), sum(nums)
3 low, high = low_high_total([60, 80, 100])
4 print(low, high)
```

报错说要拆的值太多了。函数名说了要交回三样：最低、最高、总和。点出该改的那一行。

点代码里的一行

# 自己挑战

# 最近的宝藏

你站在（0，0），只能横着或竖着走，走到（x，y）要 |x| + |y| 步。第一行读入宝藏个数 n（1 到 50），接下来 n 行每行两个整数 x 和 y（-100 到 100），用空格隔开。

找出最近的宝藏，一样近时选先出现的。输出两行：它的位置，写成（1，-2）的样子；还有“几 步”。

依次输入

```
3
3 4
-1 2
5 0
```

输出

```
(-1, 2)
3 步
```

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　每行用 x, y = input().split() 拆开，再用 int() 变成整数，打包成（x, y）放进列表。

**提示 2**　先当第一个最近：best = points[0]。再一个个比，步数更少才换，一样近不换。

**提示 3**　步数用第 16 课的 abs：abs(p[0]) + abs(p[1])。print(best) 会直接输出（x, y）的样子。

# # 造一个你的元组小程序

找一些总是成对、成组出现的东西，打成一包。

## 先做到

写一个函数，一次交回两个结果，比如一个数的两倍和一半。

## 选你喜欢的

朋友生日表：名字、月、日；
地图上的地点：名字和坐标；
单词和意思。

## 还想更厉害

读入生日表，找出这个月过生日的人；或者算两个点之间要走几步。

没有标准答案。先说清它要做什么，再试两组输入，一组试边界。

# # 和同桌换着玩

## 1

### 换着运行

运行同桌的程序，看成对的东西有没有错位

## 2

### 说一个喜欢的

告诉同桌，哪一对用得最巧

## 3

### 提一个想法

只有一组数据时，程序还对吗？

听完再改一版，让程序更稳。

# 说得清吗?

| | | |
|---|---|---|
| Q1 | **元组和列表有什么不同?** | 元组用圆括号，打好包就不能给某一格重新赋值。 |
| Q2 | **什么是拆包?** | `x, y = p`，按顺序拆成几个变量，左右个数要一样。 |
| Q3 | **函数怎么交回两个值?** | `return a, b` 打成一包，调用时 `x, y = 函数()` 拆开。 |

# 速查卡

| | |
|---|---|
| `p = (3, 5) p[0]` | 打包；用下标看 |
| `x, y = p` | 拆包，个数要一样 |
| `return a, b` | 函数一次交回两个 |
| `for name, score in pairs:` | 一对一对拿，顺手拆开 |
| `(5,)` | 只有一个值的元组要带逗号 |

# # 这些后面再学

### dict.items()

字典也能一对一对地拿出键和值

### 按第二项排序

让成绩表按分数从高到低排

### 元组当字典的键

用 (x, y) 记住地图上每一格放了什么

今天学会了把几个值打成一包。成对的东西放在一起，就不容易乱。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 **python-for-youth**

回到课程首页

这门课的路线走完了，了不起！